Table des matières

Labo 07 OLAP — Sous-requêtes, corrélation et ensembles sur l'entrepôt ..... 1
Le terrain, en une minute ..... 1
Partie 1 — Les trois formes, et la corrélation ..... 2
Partie 2 — Les questions que le GROUP BY ne sait pas poser ..... 4
Partie 3 — Ce que le contraste vous a montré ..... 8
Et ensuite ? ..... 9

# Labo 07 OLAP — Sous-requêtes, corrélation et ensembles sur l'entrepôt

*Laboratoire du Cours 07, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé*

Ce laboratoire se fait dans la ligne de commande DuckDB, sur l'entrepôt monté au Cours 00. Comptez trois quarts d'heure. Tapez les requêtes, ne les copiez pas : c'est en tapant qu'on remarque ce qu'on n'a pas compris.

Les deux voies — Ce labo a un jumeau, `Labo_07_OLTP_SQL.md`, qui pose les mêmes questions sur la base transactionnelle dans PostgreSQL. Faites celui-ci d'abord si vous venez du monde analytique, ou l'autre d'abord si vous venez du développement applicatif. Faites les deux dans tous les cas : c'est le contraste qui enseigne.

## Le terrain, en une minute

Quatre tables suffisent pour tout ce module.

| Table | Ce qu'elle contient | Colonnes utiles ici |
|---|---|---|
| `FactInternetSales` | une ligne par ligne de commande en ligne | `ProductKey`, `CustomerKey`, `SalesOrderNumber`, `OrderQuantity`, `SalesAmount`, `OrderDateKey` |
| `DimProduct` | le catalogue, en dimension à évolution lente | `ProductKey`, `EnglishProductName`, `ListPrice`, `ProductSubcategoryKey` |
| `DimProductSubcategory` | les sous-catégories | `ProductSubcategoryKey`, `EnglishProductSubcategoryName`, `ProductCategoryKey` |



© 2026 iAlexMG. Tous droits réservés.

| Table | Ce qu'elle contient | Colonnes utiles ici |
|---|---|---|
| `DimProductCategory` | les quatre catégories | `ProductCategoryKey`, `EnglishProductCategoryName` |

Posez le contexte une fois pour toutes :

```
USE dbo;
```

Une particularité de l'entrepôt qu'il faut connaître avant de commencer : `DimProduct` compte **606 lignes pour 504 produits**, parce que certains produits y ont plusieurs versions historiques.

Attention à ce que cela fait et à ce que cela ne fait pas. Une jointure du fait vers la dimension sur `ProductKey` ne double rien : cette clé désigne une version précise, et chaque ligne de vente n'en rencontre qu'une. Le vrai effet est ailleurs, et il est plus sournois. Une anti-jointure qui cherche les produits jamais vendus travaille elle aussi au grain de la version : elle liste comme « jamais vendues » 19 lignes qui sont l'ancienne version d'un produit bel et bien vendu sous sa version courante, soit 11 noms de produits. Un rapport qui titrerait « onze produits n'ont jamais trouvé preneur » serait faux, et aucune requête ne l'aurait signalé.

La leçon est celle du grain, encore : `COUNT(DISTINCT …)` sur la clé métier plutôt que sur la clé de version est ce qui ramène le compte à la question posée. Le Cours 09 traite le sujet en profondeur ; ici, on s'en sert pour comprendre pourquoi ce `DISTINCT` n'est pas une précaution de style.

---

## Partie 1 — Les trois formes, et la corrélation

### 1.1 La sous-requête scalaire

*Théorie : `Theorie_07_SQL.md`, section 2.*

Trouvez les produits dont le prix de liste dépasse la moyenne du catalogue.

```
SELECT p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
WHERE  p.ListPrice > (SELECT AVG(ListPrice) FROM DimProduct)
ORDER  BY p.ListPrice DESC;
```

Attendu : une centaine de lignes, les vélos en tête.

Maintenant la même chose, mais avec l'écart affiché. Comparez les deux écritures :

```
-- Version sous-requête : la table est relue pour calculer la moyenne.
SELECT p.EnglishProductName,
       p.ListPrice,
       p.ListPrice - (SELECT AVG(ListPrice) FROM DimProduct) AS ecart
```



© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   DimProduct AS p;

-- Version fenêtre du Cours 04 : un seul passage.
SELECT p.EnglishProductName,
       p.ListPrice,
       p.ListPrice - AVG(p.ListPrice) OVER () AS ecart
FROM   DimProduct AS p;
```

Le résultat est identique. Préfixez chacune des deux requêtes de `EXPLAIN` et comptez les `SEQ_SCAN` de `DimProduct` : deux pour la sous-requête, reliés par un `CROSS_PRODUCT`, un seul pour la fenêtre, suivi d'un opérateur `WINDOW`. C'est le genre de vérification que le Cours 10 systématisera.

### 1.2 La sous-requête corrélée

*Théorie : `Theorie_07_SQL.md`, section 3.*

Pour chaque produit, comptez ses lignes de vente.

```sql
SELECT p.EnglishProductName,
       (SELECT COUNT(*)
        FROM   FactInternetSales AS f
        WHERE  f.ProductKey = p.ProductKey) AS nb_lignes
FROM   DimProduct AS p
ORDER  BY nb_lignes DESC
LIMIT  10;
```

Attendu : une dizaine de lignes, les accessoires en tête (ils se vendent en grand nombre, pas en grande valeur).

Reprenez la requête en la préfixant de `EXPLAIN`. Vous n'y trouverez pas de boucle : le plan affiche un `LEFT_DELIM_JOIN`, un `HASH_JOIN` et un `HASH_GROUP_BY`. DuckDB a décorrélé la requête et l'a transformée en jointure avec agrégation, ce que la section 3 de la théorie annonce. Le mot du plan, *delim join*, est le nom technique de cette transformation.

### 1.3 La sous-requête de table

*Théorie : `Theorie_07_SQL.md`, section 2.*

```sql
SELECT s.ProductKey, s.qte_totale
FROM  (SELECT ProductKey, SUM(OrderQuantity) AS qte_totale
       FROM   FactInternetSales
       GROUP  BY ProductKey) AS s
WHERE s.qte_totale > 1000
ORDER BY s.qte_totale DESC;
```

Réécrivez-la en CTE, comme au Cours 05, et gardez la version que vous trouvez la plus lisible. La réponse attendue est la CTE : elle se lit de haut en bas.



© 2026 iAlexMG. Tous droits réservés.

## Partie 2 — Les questions que le `GROUP BY` ne sait pas poser

### 2.1 Les produits jamais vendus, par anti-jointure

*Théorie : `Theorie_07_SQL.md`, section 4.*

```sql
SELECT p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
WHERE  NOT EXISTS (SELECT 1
                   FROM   FactInternetSales AS f
                   WHERE  f.ProductKey = p.ProductKey)
ORDER  BY p.EnglishProductName;
```

Attendu : plusieurs centaines de lignes. Beaucoup de produits de l'entrepôt ne sont vendus qu'aux revendeurs, donc absents de `FactInternetSales`.

Écrivez maintenant la même question avec `NOT IN`, puis avec un `LEFT JOIN` :

```sql
-- Avec NOT IN.
SELECT COUNT(*)
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (SELECT f.ProductKey FROM FactInternetSales AS f);

-- Avec LEFT JOIN et test d'absence.
SELECT COUNT(*)
FROM   DimProduct AS p
LEFT   JOIN FactInternetSales AS f ON f.ProductKey = p.ProductKey
WHERE  f.ProductKey IS NULL;
```

Les trois comptes doivent coïncider ici, parce que `FactInternetSales.ProductKey` ne contient pas de `NULL`. Gardez ce résultat en tête : la sous-partie 2.2 vous fera fabriquer le cas où ils divergent.

L'entrepôt a une quatrième écriture, en toutes lettres :

```sql
SELECT COUNT(*) AS jamais_vendus
FROM   DimProduct AS p
ANTI   JOIN FactInternetSales AS f ON f.ProductKey = p.ProductKey;

SELECT COUNT(*) AS vendus
FROM   DimProduct AS p
SEMI   JOIN FactInternetSales AS f ON f.ProductKey = p.ProductKey;
```



© 2026 iAlexMG. Tous droits réservés.

Attendu : 448 et 158, soit les 606 lignes de `DimProduct`. `SEMI JOIN` garde chaque produit une seule fois, là où une jointure interne en rendrait une ligne par vente. Ne gardez cette forme que pour du code qui reste sur DuckDB : le labo jumeau montre ce qu'en fait PostgreSQL.

### 2.2 Le piège `NOT IN`, provoqué exprès

*Théorie : `Theorie_07_SQL.md`, section 5.*

Fabriquez une liste qui contient un `NULL` et observez.

```sql
-- Combien de produits ne sont pas dans la liste (1, 2) ?
SELECT COUNT(*) AS avec_valeurs
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (1, 2);

-- La même question, avec un NULL ajouté à la liste.
SELECT COUNT(*) AS avec_null
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (1, 2, NULL);
```

Attendu : la première renvoie 604, la seconde renvoie **0**. Aucune erreur, aucun avertissement. C'est le résultat le plus important du laboratoire. Refaites-le avec `NOT EXISTS` sur une sous-requête qui renvoie ces mêmes valeurs, et constatez que le compte reste juste.

### 2.3 Les clients d'une catégorie mais pas d'une autre

*Théorie : `Theorie_07_SQL.md`, section 6.*

On utilise ici les opérateurs d'ensembles. Commencez par la vue de travail :

```sql
WITH ventes_par_categorie AS (
    SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie
    FROM   FactInternetSales     AS f
    JOIN   DimProduct            AS p  ON p.ProductKey = f.ProductKey
    JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
    JOIN   DimProductCategory    AS c  ON c.ProductCategoryKey = sc.ProductCategoryKey
)
SELECT CustomerKey FROM ventes_par_categorie WHERE categorie = 'Bikes'
EXCEPT
SELECT CustomerKey FROM ventes_par_categorie WHERE categorie = 'Accessories';
```

Attendu : quelques milliers de clients. Ce sont ceux qui ont acheté un vélo sans jamais prendre d'accessoire, une population que le service commercial voudra sûrement rappeler.



© 2026 iAlexMG. Tous droits réservés.

Remplacez `EXCEPT` par `INTERSECT` pour obtenir les clients des deux catégories, puis par `UNION ALL` et comparez le nombre de lignes avec `UNION`. L'écart entre les deux derniers est exactement le nombre de doublons que `UNION` a supprimés, et ce dédoublonnage a un coût, que le Cours 10 apprend à lire dans un plan.

### 2.4 Le top 3 par sous-catégorie, en trois écritures

*Théorie : `Theorie_07_SQL.md`, sections 2 et 7.*

D'abord avec `LATERAL` :

```sql
SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
       top.EnglishProductName,
       top.ListPrice
FROM   DimProductSubcategory AS sc,
       LATERAL (SELECT p.EnglishProductName, p.ListPrice
                FROM   DimProduct AS p
                WHERE  p.ProductSubcategoryKey = sc.ProductSubcategoryKey
                ORDER  BY p.ListPrice DESC
                LIMIT  3) AS top
ORDER  BY sous_categorie, top.ListPrice DESC;
```

Puis avec `QUALIFY`, la forme la plus courte :

```sql
SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
       p.EnglishProductName,
       p.ListPrice
FROM   DimProduct AS p
JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
QUALIFY ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                           ORDER BY p.ListPrice DESC) <= 3
ORDER  BY sous_categorie, p.ListPrice DESC;
```

Puis avec une fenêtre et une CTE, la forme portable :

```sql
WITH classe AS (
    SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
           p.EnglishProductName,
           p.ListPrice,
           ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                              ORDER BY p.ListPrice DESC) AS rang
    FROM   DimProduct AS p
    JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
)
SELECT sous_categorie, EnglishProductName, ListPrice
FROM   classe
```



© 2026 iAlexMG. Tous droits réservés.

```sql
WHERE  rang <= 3
ORDER  BY sous_categorie, ListPrice DESC;
```

Les trois donnent le même résultat. Notez que `LATERAL` conserve les sous-catégories sans produit seulement si on écrit `LEFT JOIN LATERAL … ON true` ; avec la virgule, elles disparaissent. Les deux autres écritures les perdent aussi, puisqu'elles partent de `DimProduct`.

### 2.5 Une anti-jointure de contrôle qualité

*Théorie : `Theorie_07_SQL.md`, section 4.*

Voici l'usage professionnel de l'anti-jointure : chercher les lignes de fait dont la dimension est absente. Ce sont les erreurs d'alimentation les plus coûteuses, parce qu'elles font disparaître du chiffre d'affaires d'un rapport sans laisser de trace.

```sql
SELECT COUNT(*) AS lignes_orphelines
FROM   FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey);
```

Attendu : zéro. AdventureWorks DW est un jeu de démonstration propre sur ce point. Un zéro n'est pas une déception : c'est le résultat qu'on espère, et c'est le contrôle qu'on met en place **avant** d'en avoir besoin. Refaites-le sur `CustomerKey` et sur `OrderDateKey`.

Contrôle de cohérence — Gardez cette requête. C'est le geste qu'un ingénieur analytique répète partout : « ai-je des transactions qui référencent un instrument inconnu ? », « des ventes dont le magasin n'existe plus dans le référentiel ? ». La forme ne change jamais, seules les tables changent.

### 2.6 Rapprocher deux ensembles, dans les deux sens

*Théorie : `Theorie_07_SQL.md`, section 6.*

Les clients de 2012 et de 2013, comparés dans les deux directions :

```sql
WITH clients AS (
    SELECT DISTINCT CustomerKey, YEAR(OrderDate) AS annee
    FROM   FactInternetSales
)
SELECT (SELECT COUNT(*) FROM clients WHERE annee = 2012) AS en_2012,
       (SELECT COUNT(*) FROM clients WHERE annee = 2013) AS en_2013,
       (SELECT COUNT(*) FROM (SELECT CustomerKey FROM clients WHERE annee = 2012
                              EXCEPT
                              SELECT CustomerKey FROM clients WHERE annee = 2013)) AS perdus,
       (SELECT COUNT(*) FROM (SELECT CustomerKey FROM clients WHERE annee = 2013
                              EXCEPT
                              SELECT CustomerKey FROM clients WHERE annee = 2012)) AS gagnes;
```

Attendu : 3 255 clients en 2012, 17 429 en 2013, 287 perdus et 14 461 gagnés. Chaque direction répond à sa moitié de la question, et aucune ne se déduit de l'autre.



© 2026 iAlexMG. Tous droits réservés.

Fabriquez maintenant une copie fautive : les ventes du 1er janvier 2013, dont les premières lignes de chaque commande sont chargées deux fois.

```sql
WITH source AS (
    SELECT SalesOrderNumber, SalesOrderLineNumber, SalesAmount
    FROM   FactInternetSales
    WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2013-01-02'
),
cible AS (
    SELECT * FROM source
    UNION ALL
    SELECT * FROM source WHERE SalesOrderLineNumber = 1
)
SELECT (SELECT COUNT(*) FROM source) AS lignes_source,
       (SELECT COUNT(*) FROM cible)  AS lignes_cible,
       (SELECT COUNT(*) FROM (SELECT * FROM cible  EXCEPT     SELECT * FROM source)) AS en_trop_except,
       (SELECT COUNT(*) FROM (SELECT * FROM source EXCEPT     SELECT * FROM cible))  AS manquantes_except,
       (SELECT COUNT(*) FROM (SELECT * FROM cible  EXCEPT ALL SELECT * FROM source)) AS en_trop_except_all;
```

Attendu : 39 lignes à la source, 54 à la copie, 0 et 0 par `EXCEPT`, 15 par `EXCEPT ALL`. `EXCEPT` compare des ensembles de valeurs, et la copie a bien les mêmes valeurs : il conclut à l'égalité. `EXCEPT ALL` compte les occurrences et trouve les 15 lignes en trop. Un rapprochement de lignes s'écrit donc en `EXCEPT ALL`, dans les deux sens.

Dernier point, les `NULL` :

```sql
SELECT COUNT(*) AS reste
FROM (SELECT 1 AS a, NULL AS b
      EXCEPT
      SELECT 1, NULL);
```

Attendu : 0. Pour un opérateur d'ensemble, deux `NULL` se valent, et la ligne s'annule. Une anti-jointure écrite avec `=` sur la colonne `b` l'aurait gardée, puisque `NULL = NULL` vaut inconnu.

---

## Partie 3 — Ce que le contraste vous a montré

*Théorie : `Theorie_07_SQL.md`, section 8.*

Vous verrez la colonne PostgreSQL se remplir en faisant `Labo_07_OLTP_SQL.md`. Notez déjà ce que l'entrepôt vous a appris.



© 2026 iAlexMG. Tous droits réservés.

| Observation | Sur l'entrepôt |
|---|---|
| Le top N par groupe | `QUALIFY` en fait la forme la plus courte des deux terrains, en une seule requête sans CTE |
| `LATERAL` | s'écrit avec une virgule, sans le mot-clé `JOIN`, mais perd alors les lignes de la table de gauche sans correspondance |
| `EXCEPT` | `EXCEPT ALL` existe et compte les répétitions ; `MINUS`, le synonyme d'Oracle et de Snowflake, rend une erreur de syntaxe |
| La semi- et l'anti-jointure | s'écrivent aussi `SEMI JOIN` et `ANTI JOIN`, une forme que PostgreSQL ne connaît pas |
| Le piège `NOT IN` | se démontre sur une liste littérale, faute de colonne nullable évidente dans le fait |
| La corrélation | `EXPLAIN` prouve la décorrélation : aucune boucle dans le plan |

---

## Et ensuite ?

Vous savez maintenant poser les questions négatives et comparatives, et aller chercher les N premiers de chaque groupe. Le `LATERAL` que vous venez d'écrire est le geste exact de l'as-of join du Cours 09 : au lieu de « les trois plus chers de cette sous-catégorie », vous demanderez « le dernier prix connu avant cette date ». La forme sera identique.

Avant d'y aller, deux étapes : le Cours 08 sur les données semi-structurées, où `LATERAL` sert à aplatir un tableau JSON, et les exercices de ce module, dans `Exercices_corriges_07_SQL.md`.



© 2026 iAlexMG. Tous droits réservés.